Health Companion — Technical Documentation

Oct 13, 2026 · @Dheepthi Reddy

1. Overview

Health Companion is a two-role web app: patients log their daily health in about a minute, and clinicians monitor every patient, message them privately and generate guard-railed AI summaries. It is a Next.js frontend talking to a FastAPI backend over a single `/api` proxy, with all data in one SQLite file.

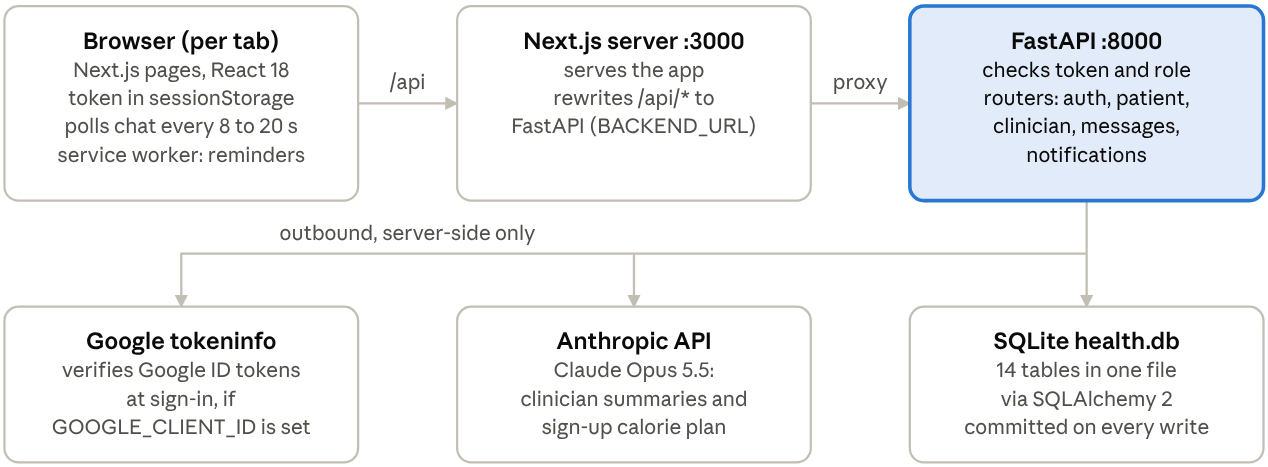
Patients (route `/`): Today dashboard with an animated check-in streak, a combined Track page (food, activity, vitals, BMI), medicines with reminders, habits, profile and settings, and a floating contact button for private chats with their doctors.

Clinicians (route `/clinician`): a triaged patient list (their own patients first), full 30-day records per patient, an AI summary button, availability and on-call settings, and a collapsible chat dock.

Request flow

1. The browser calls `/api/...` on the Next.js server (port 3000).
2. `next.config.mjs` rewrites `/api/:path*` to the FastAPI backend (`BACKEND_URL`, default `http://127.0.0.1:8000`), so no CORS is needed in normal use.
3. FastAPI validates the body with Pydantic (`app/schemas.py`), checks the bearer token and role (`app/security.py`), and reads or writes rows through SQLAlchemy 2 (`app/models.py`).
4. Rows live in `backend/health.db` and are committed immediately.
5. Two features call the Anthropic API (Claude Opus 5.5) from the backend only: clinician summaries and the registration calorie plan. The browser never talks to Claude directly.

The browser only talks to Next.js; FastAPI owns the data and every outside call



system architecture · 6 parts

Only FastAPI holds secrets (signing key, Anthropic key) and touches the database; the browser never calls Claude, Google's token check or SQLite directly.

2. Tech stack and repository layout

The backend is Python 3.9+ with FastAPI, SQLAlchemy 2, Pydantic 2, SQLite and the `anthropic` SDK (1.13); the frontend is Next.js 16 (App Router), React 18, TypeScript, Tailwind 4 and vitest. There are no UI or animation libraries: icons, animations, toasts and sounds are hand-written.

Backend (`backend/`)

Path	Purpose
`app/main.py`	App factory; startup creates tables, adds missing columns, backfills message conversations, seeds demo data
`app/database.py`	Engine, session factory, `get_db` , `add_missing_columns()` lightweight migration
`app/models.py`	SQLAlchemy tables (users, readings, food, workouts, medicines, habits, check-ins, messages, logs)

Path	Purpose
`app/schemas.py`	Pydantic request/response models and per-role field sets
`app/security.py`	PBKDF2 password hashing, HMAC-signed tokens, `current_user`, `require_patient`, `require_clinician`
`app/care_team.py`	Doctor availability, on-call cover, auto-assignment, validating a patient's doctor choice
`app/calories.py`	Free-text meal parser, calories burned, BMI, energy balance and tips
`app/calorie_plan.py`	AI-assisted starting calorie goal at registration
`app/llm.py`	AI clinician summaries with guardrails
`app/google_auth.py`	Google ID-token verification
`app/stats.py`	Aggregations used by the home card, clinician list and AI summaries
`app/streaks.py`	Check-in streak, longest streak, milestones
`app/alerts.py`	BP and glucose status and severity
`app/suggestions.py`	Personal nudge text for the login reminder
`app/seed.py`	Turns `app/data/demo_data.json` into 90 days of demo rows
`app/data/*.json`	Static data: foods, activities, starter habits, demo clinic and logins
`app/routers/auth.py`	Login, register, Google, clinician directory, profile (`/me`)
`app/routers/patient.py`	Home, check-in, vitals, BMI, food, energy, activity, medicines, habits
`app/routers/clinician.py`	Patient list, records, AI summary, chat threads and messages
`app/routers/messages.py`	Patient-side private chats, contact rules, legacy backfill

Path	Purpose
`app/routers/notifications.py`	Due medicine reminders and the login nudge
`tests/`	pytest suites (67 tests)

Frontend (`frontend/`)

Path	Purpose
`app/layout.tsx` , `app/globals.css`	Root layout, Plus Jakarta Sans font, base styles, reduced-motion rules
`app/login/page.tsx`	Sign in / Create account, patient vs clinician login, doctor picker, body metrics, Google button
`app/page.tsx`	Patient Today: plan card, streak, status, rings, tips
`app/track/page.tsx`	Food, activity, energy overview, vitals, BMI gauge
`app/medicines/page.tsx` , `app/habits/page.tsx`	Dose schedule and habit counters
`app/profile/page.tsx` , `app/settings/page.tsx`	Shared by both roles; role-specific cards
`app/clinician/page.tsx` , `app/clinician/patients/[id]/page.tsx`	Patient list and one patient's records
`components/AuthGate.tsx`	Loads the user, role routing, picks the shell
`components/PatientShell.tsx` , `ClinicianShell.tsx` , `TopBar.tsx` , `ProfileMenu.tsx`	Layout chrome
`components/ContactFab.tsx` , `ClinicianChatDock.tsx` , `ChatThread.tsx`	Messaging UI

Path	Purpose
`components/StreakHero.tsx`, `Burst.tsx`, `PlanWelcome.tsx`, `DoctorPicker.tsx`, `GoogleSignIn.tsx`, `GoalInput.tsx`, `SummaryBox.tsx`, `Toast.tsx`, `Notifier.tsx`, `Backdrop.tsx`	Feature components
`components/ui.tsx`, `components/icons.tsx`	Design-system primitives and the SVG icon set
`lib/api.ts`	Fetch wrapper, per-tab token storage, `useFetch`, date helpers
`lib/types.ts`	TypeScript shapes of every API response
`lib/prefs.ts`, `lib/sound.ts`, `lib/password.ts`, `lib/emergency.ts`	Device preferences, synthesized sounds, password strength, emergency number
`public/sw.js`	Service worker that shows notifications

3. Running locally and configuration

Two processes run side by side: FastAPI on port 8000 and Next.js on port 3000; the first backend start creates and seeds `backend/health.db`.

```
# Backend (http://127.0.0.1:8000, API docs at /docs)
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload

# Frontend (http://localhost:3000)
cd frontend
npm install
npm run dev
```

On every backend start (`lifespan` in `app/main.py`): `create_all` makes missing tables, `add_missing_columns()` adds new nullable columns to existing tables, `messages.backfill()` assigns old messages to a conversation, and `seed_demo_data()`

fills the demo clinic only if the database has no users. Restarting never wipes data; `python -m app.seed --reset` drops everything and reseeds.

Variable	Where	Default	Effect
`DATABASE_URL`	backend	`sqlite:///./health.db`	Database location (tests point it at a temp file)
`SECRET_KEY`	backend	`dev-only-change-me`	HMAC key that signs login tokens; must be set for any real deployment
`SEED_DEMO`	backend	`1`	`0` starts with an empty database
`ANTHROPIC_API_KEY` or `ANTHROPIC_AUTH_TOKEN`	backend	unset	Enables AI summaries and AI calorie plans; without it both use their non-AI fallbacks
`LLM_MODEL`	backend	`claude-opus-5-5`	Model for both AI features
`GOOGLE_CLIENT_ID`	backend	unset	Enables Sign in with Google; the frontend reads it from `/api/auth/google-config`
`BACKEND_URL`	frontend	`http://127.0.0.1:8000`	Where Next.js forwards `/api/*`

Login tokens last 12 hours (`TOKEN_TTL_SECONDS`). Passwords use PBKDF2-SHA256 with 100,000 iterations and a random salt per user.

4. Data model

All data is in 14 SQLite tables in `backend/health.db` ; patients and clinicians share the `users` table, and every patient record carries a `user_id` that every query filters on. Deleting a user cascades to their records (SQLite foreign keys are switched on per

connection).

Table	Columns	Notes
`users`	id, email (unique), password_hash, name, role (`patient` / `clinician`), phone, clinician_id → users, age, emergency_number, calorie_goal (1800), activity_goal_min (30), notifications_enabled, gender, height_cm, blood_type, allergies, emergency_contact_name, emergency_contact_phone, bio, available, on_call	`clinician_id` = a patient's chosen doctor; `available` / `on_call` are clinician-only (null counts as available / not on call)
`bp`	id, user_id, measured_at, systolic, diastolic, pulse	Status and severity computed on read (`app/alerts.py`)
`glucose`	id, user_id, measured_at, value (mg/dL), context (`fasting`, `before_meal`, `after_meal`, `bedtime`)	Status and severity computed on read
`bmi`	id, user_id, measured_at, height_cm, weight_kg, bmi, category	First row written at registration; latest weight feeds calories burned
`medications`	id, user_id, name, dose, times (comma-separated `HH:MM`), active	Each time is one daily dose slot
`medication_logs`	id, user_id, medication_id, day, slot, status (`taken` / `skipped`)	Unique per (medication, day, slot)
`food_entries`	id, user_id, day, meal_type, name, quantity, calories, carbs_g, protein_g, fat_g, sodium_mg, source (`text` / `one_tap`)	`one_tap` = the small gap filled by "I matched my goal"
`workouts`	id, user_id, day, kind, duration_min, intensity, calories_burned	Calories computed at insert
`habits`	id, user_id, name, target, unit	New patients get 3 starter habits

Table	Columns	Notes
`habit_logs`	id, user_id, habit_id, day, count	Unique per (habit, day); done when count ≥ target
`checkins`	id, user_id, day	Unique per (user, day); streaks are derived from these
`messages`	id, patient_id, clinician_id, sender_id, body, created_at	(patient_id, clinician_id) = one private conversation
`notification_log`	id, user_id, kind, key, last_sent_at	Unique per (user, kind, key); drives the 30-minute repeat
`summary_audit`	id, clinician_id, patient_id, days, source (`llm` / `template`), guardrail_notes, created_at	One row per AI summary request

Migrations. There is no migration framework. `add_missing_columns()` compares each model with the live table and runs `ALTER TABLE … ADD COLUMN` for any missing nullable column, which is why every column added after launch (profile fields, `available`, `on_call`, `messages.clinician_id`) is nullable. A new non-nullable column or a type change would need a real migration tool such as Alembic.

Backfill. `messages.backfill()` gives pre-existing messages their conversation: a clinician's message belongs to its sender; a patient's message belongs to the clinician who wrote most recently before it in that patient's history, else the patient's own doctor. It only writes when something was missing, so it is safe on every start.

5. Authentication, roles and sessions

Every login must state its role, and an account only works on its own role's login; after that, every API route checks the token and the role again. Patients can self-register; clinician accounts exist only through the clinic (seed data).

Tokens

- A token is `base64(json {uid, exp})` + `.` + an HMAC-SHA256 signature of that JSON with `SECRET_KEY`, valid for 12 hours (`app/security.py`). It is stateless: nothing is stored server-side.

- The browser sends it as `Authorization: Bearer <token>`. `current_user` verifies the signature and expiry and loads the user; `require_patient` / `require_clinician` reject the other role with 403.
- A 401 anywhere makes the frontend clear the tab's token and go to `/login` (`lib/api.ts`).

Login (`POST /api/auth/login`)

1. Body: `email`, `password`, `role` (`patient` or `clinician`). `role` is required: a missing or invalid role is rejected with 422, so no client (including the `/docs` page) can skip the check.
2. Wrong email or password → 401 "Wrong email or password" (the same message for both, so emails cannot be probed).
3. Right credentials, wrong role → 403 "This is a patient account. Please use Patient login." (or the clinician version).
4. Success → `{token, user}`.

Registration (`POST /api/auth/register`, patients only)

Field	Rule
name	2–80 characters after trimming
email	Must look like an address; stored lower-case; 409 if it already exists
password	8–128 characters (the form shows a strength meter, `lib/password.ts`)
age	13–120; the form requires it (the server defaults to 35 for the calorie formula if absent)
height_cm, weight_kg	Required; 100–250 cm and 25–300 kg
activity_level	`sedentary`, `light` (default), `moderate`, `active`
phone, gender	Optional; empty strings are stored as null
clinician_id	Optional; 0 or absent = any available doctor

The server then: resolves the doctor (section 7), asks the calorie planner for a starting goal (section 9), creates the user with that goal and 3 starter habits, saves a first BMI row, and returns `{token, user, plan}`. The frontend stores `plan` in `sessionStorage` for the one-time "Your starting plan" card.

Sign in with Google (`POST /api/auth/google`)

1. The page fetches `/api/auth/google-config`; with no `GOOGLE_CLIENT_ID` the button is shown disabled with an explanation.
2. Google Identity Services renders Google's own button and returns an ID token (popup mode).
3. The backend sends the token to Google's `tokeninfo` endpoint and accepts it only if `aud` equals our client ID, `iss` is Google and `email_verified` is true (`app/google_auth.py`).
4. An existing account signs in by matching email, subject to the same role check. A new address on the patient login creates a patient (auto-assigned doctor, starter habits, unusable random password). A new address on the clinician login is refused: nobody can make themselves a clinician.

Per-tab sessions

The token lives in `sessionStorage` (one per tab), so a patient and a clinician can be signed in side by side in two tabs. The latest sign-in is also copied to `localStorage` so a newly opened tab starts signed in. Signing out clears the tab's token and removes the shared copy only if it was this tab's. The login page stays reachable while signed in; it shows "Signed in as …" with a Continue button only when the selected login matches that account's role.

Routing (`components/AuthGate.tsx`)

- Not signed in → `/login`. Clinicians are kept under `/clinician`; patients are kept out of it. `/profile` and `/settings` are shared by both roles.
- The gate loads `/api/auth/me` once, then renders the patient or clinician shell.

Profile updates (`PATCH /api/auth/me`)

Who	May change
Anyone	name, phone, age, gender, height_cm, blood_type, allergies, emergency contact name and phone, bio
Patients only	emergency_number, calorie_goal (800–4500), activity_goal_min (0–300), notifications_enabled, clinician_id
Clinicians only	available, on_call

A field outside the caller's set → 403. An emptied text field is stored as null (name is required). `clinician_id` goes through the same doctor resolution as registration.

6. Patient features

Everything a patient logs is computed server-side from rows they own; the frontend only displays it. Dates are always the browser's local date (`localDate()` in `lib/api.ts`), passed as `?day=YYYY-MM-DD`, so "today" never drifts to UTC.

Today (`/`)

- **Starting plan card** (once, after sign-up): goal, BMI, estimated needs, the reason, and whether AI or the formula set it.
- **Streak hero:** current streak, the last 7 days, progress to the next milestone and the check-in button (animation in section 12).
- **Status card** from the latest BP and glucose readings within 36 hours: none → amber "No readings in the last day"; worst severity ⩾ 2 → red with that reading's message; severity 1 → amber; otherwise green.
- **Rings:** net calories against goal, active minutes, medicines taken today (with the next pending dose), habits done.
- **Tips** from the energy balance (below).

Streaks (`app/streaks.py`)

- `POST /api/checkin` adds one row per day (idempotent).

- Current streak = consecutive check-in days ending today, or ending yesterday if today is not checked in yet (the streak stays alive until midnight). Longest streak = the longest run ever.
- Every 5th consecutive day is a milestone (bigger burst and a fanfare in the UI).

Track (`/track`) — food, activity and vitals on one page

Food parsing (`app/calories.py` , `POST /api/food/preview` then `POST /api/food`):

1. Lower-case the text and split on commas, semicolons, `+` , `&` , "and", "with", "plus".
2. For each chunk read a quantity: a number (`2` , `1.5` , `1/2`) or a number word (`a` , `two` , `half` , `couple`); default 1.
3. Drop filler words (`cup` , `bowl` , `ate` , `had` , …), singularise the rest and match the longest alias in `app/data/foods.json` (42 foods with serving size, kcal, carbs, protein, fat, sodium).
4. Multiply the per-serving values by the quantity (allowed 0–20). Unmatched chunks are returned so the UI can say "check spelling" before anything is saved.

Activity: calories burned = MET × intensity factor × weight kg × hours. MET values (12 activities) and factors (light 0.8, moderate 1.0, vigorous 1.25) are in `app/data/activities.json` ; weight comes from the latest BMI row, else 70 kg.

Energy balance (`GET /api/energy`):

```
\text{net} = \text{eaten} - \text{burned}, \qquad \text{difference} =
\text{net} - \text{goal}
```

- State is `matched` when the difference is within 10% of the goal, `over` above it, `under` below it.
- Tips: over → the minutes of brisk walking that would burn the excess, plus a lighter-next-meal tip; under → a balanced-snack tip; short of the activity goal → how many minutes are left.
- `goal_met` is true only when the state is `matched` and active minutes ≥ the activity goal; `goal_missing` lists what is left.

"I matched my goal" (`POST /api/food/matched-goal`): refused with 409 "Not yet: …" unless `goal_met` . When allowed, the small gap still inside the tolerance is recorded as one self-declared `one_tap` snack so the day closes on target. The button is disabled in the UI with a two-item checklist until both goals are met.

Vitals: BP (systolic/diastolic) and glucose with its context. Classification (`app/alerts.py`, ADA and AHA/ACC-based, informational only):

Reading	Bands (severity 0 ok → 3 urgent)
Blood pressure	Normal < 120/80 (0) · Slightly raised 120–129 (1) · Elevated Stage 1 130–139 or 80–89 (1) · High Stage 2 ≥ 140 or ≥ 90 (2) · Low < 90 or < 60 (2) · Crisis > 180 or > 120 (3)
Glucose, fasting	Very low < 54 (3) · Low < 70 (2) · In range ≤ 130 (0) · Elevated ≤ 180 (1) · High ≤ 300 (2) · Very high > 300 (3)
Glucose, after a meal	Same, with In range ≤ 180 and Elevated ≤ 250

BMI: weight / height², rounded to 0.1; categories Underweight < 18.5, Healthy range < 25, Overweight < 30, Obese range ≥ 30. The gauge spans BMI 15–40. Height is pre-filled from the profile.

Goals for calories and activity minutes are edited in place and save on blur.

Medicines (`/medicines`)

- A medicine has a name, dose and one or more daily times; each time is a dose slot.
- `GET /api/medications/schedule?day=` lists today's slots with `pending`, `taken` or `skipped`; Taken / Skip writes one `medication_logs` row per slot.
- The Reminders switch sets `notifications_enabled` (section 10).

Habits (`/habits`)

One-off habits are a done/not-done button; counted habits (for example 8 glasses of water) have − and + buttons. A habit is done when count ≥ target; reaching it plays a chime and a small burst.

Profile and settings

- **Profile:** personal details, health basics (height, blood type, allergies), My doctor picker (saves on choose), emergency contact. A completeness bar counts filled fields; a sticky bar shows unsaved changes; only changed fields are sent.

- **Settings:** sound effects and animations (device-only, `lib/prefs.ts`), medicine reminders, daily goals, local emergency number (used by "Emergency: call …" in the profile menu; default guessed from the browser locale, e.g. 911 in the US, 112 as the fallback).

7. Clinician features and the care team

A clinician lands on their own patients first, can read any patient's records, and controls whether they are available and on call; each patient has exactly one chosen doctor (`users.clinician_id`).

Patient list (`/clinician`, `GET /api/clinician/patients`)

- Per patient: age, latest BP, glucose and BMI with status, current streak, last check-in, 7-day medicine adherence (taken ÷ expected doses), assigned `clinician_id`, and `attention` = the worse severity of the latest BP and glucose.
- The server sorts by attention (highest first), then name. The page splits that list into **Your patients** (assigned to the signed-in clinician) and **Other patients in the clinic**, keeping the attention order inside each group.
- KPI tiles: total patients, needing attention (severity ≥ 2), active streaks, average 7-day adherence. Search, plus filters All / My patients / Needs attention.
- Each card has a Message button (opens the chat dock on that patient) and the AI summary control (7, 30 or 90 days; section 9).

Patient records (`/clinician/patients/[id]`)

`GET /api/clinician/patients/{id}/records?days=30`: profile and goals, streak, the last 30 days of BP, glucose, food (summed per day), activity and BMI history, shown as tables with status pills.

Choosing a doctor

Situation	Result (`app/care_team.py`)
Patient picks a doctor (sign-up or Profile)	That doctor, even if away

Situation	Result (`app/care_team.py`)
Patient picks "Any available doctor" (id 0)	The available clinician with the fewest patients (ties → lowest id); if nobody is available, any clinician
Google sign-up	Same as "Any available doctor"
Unknown id or a non-clinician	422 "That doctor could not be found"

The public directory `GET /api/auth/clinicians` (no login needed, used at sign-up) returns id, name, gender, bio, available and on_call only: no email or phone.

Availability and on-call cover

- Settings → **Available to patients** (`available`) and **On call** (`on_call`). Turning availability off also turns on-call off; turning on-call on also marks the clinician available.
- A clinician counts as on call only if they are also available.
- Every patient's home payload includes `on_call` : the on-call clinicians other than their own doctor. When their own doctor is away, the contact button says so, opens chat on the on-call doctor and offers "Call on-call doctor" with that doctor's phone.
- Demo state: Dr. Arjun Shetty is on call; Dr. Neil D'Souza is away.

8. Messaging

Every conversation is private to one patient and one clinician: a message row stores `patient_id` + `clinician_id` , and each side can only read conversations it is part of. A patient can hold several conversations (one per doctor); a clinician never sees a colleague's conversation with the same patient.

Who can see and send what

Action	Endpoint	Rule
Patient reads	`GET /api/messages`	All of their own messages, across conversations, each tagged with `clinician_id`

Action	Endpoint	Rule
Patient sends	`POST /api/messages` `{body, clinician_id?}`	Default recipient = their own doctor. Allowed recipients: their doctor, any on-call clinician, or a clinician who has already written to them. Anyone else → 403. No doctor and no `clinician_id` → 422
Clinician lists chats	`GET /api/clinician/threads`	Patients they have a conversation with, plus their own assigned patients (so a chat can be started), newest first, with the last message from **their** conversation only
Clinician reads	`GET /api/clinician/patients/{id}/messages`	Only their own conversation with that patient
Clinician sends	`POST /api/clinician/patients/{id}/messages`	Allowed to any patient; the message is filed under the sender's conversation

Message bodies are 1–1,000 characters, stored trimmed, with `created_at` set by the server.

Delivery: polling, not push

There is no WebSocket. Clients poll:

Client	Interval
Patient, chat closed	every 20 s (`/api/messages`)
Patient, chat open	every 8 s
Clinician dock list	every 20 s (`/api/clinician/threads`)
Clinician open thread	every 10 s

So a new message appears within about 8–20 seconds.

Patient chat (`components/ContactFab.tsx`)

1. A round button sits bottom-right on every patient page; hover (or tap) fans out **Text** and **Call**. When the patient's doctor is away it shows "Dr. X is away", "Message the on-call doctor" and "Call on-call doctor".
2. The contact list is built in the browser: their doctor ("Your doctor" or "Your doctor · away"), on-call doctors while their doctor is away, and every clinician who appears in their messages. The server applies the same rule when they send.
3. The chat panel shows one tab per contact with its own unread count, filters messages by `clinician_id` and sends with the selected `clinician_id`. The header says "Private conversation".
4. **Unread:** per account and per doctor, the browser remembers the newest message id seen from each doctor (`localStorage` key `hc_seen_conv_<userId>`). Unread = that doctor's messages with a higher id. Viewing a tab marks it read. The badge on the button is the total.
5. **Alerts:** when a poll brings a newer clinician message and that conversation is not on screen, a chime plays and a toast says "New message from …". The backlog on first load only shows the badge.

Clinician dock (`components/ClinicianChatDock.tsx`)

- A dark chat button bottom-right; a red badge counts conversations whose last message is from the patient ("awaiting reply").
- The panel lists threads with search and filters All / Mine (patients assigned to them) / Awaiting. "Mine" and "Awaiting" hide other threads by design.
- Clicking a thread opens it; "View records →" links to the patient. The Message buttons on the patient list and records page open the dock on that patient, passing the name so a brand-new conversation still shows it.

Shared message UI (`components/ChatThread.tsx`)

Bubbles grouped by day, sender name on incoming messages, time on each, auto-scroll to the newest, a composer where Enter sends and Shift+Enter adds a line, and a soft sound on send.

History before private conversations

Messages written when there was one shared thread per patient were assigned on startup by `messages.backfill()` (section 4). In the current database all three older messages belong to Dr. Kavitha Menon's conversations.

9. How the AI works

Two features use Claude (model `claude-opus-5-5`, overridable with `LLM_MODEL`), and both are built so the AI can only phrase or pick within numbers the app computed itself; if the AI is missing, refuses, errors or breaks a rule, a deterministic non-AI result is shown instead and labelled as such. Calls go from the backend through the official `anthropic` Python SDK; the SDK is imported lazily, so the app runs without it or without a key.

A. Clinician summaries (`app/llm.py`, `POST /api/clinician/patients/{id}/summary?days=7|30|90`)

Input. `app/stats.py` computes period statistics for the patient: BP and glucose averages, ranges and counts by status, latest BMI and category, medicine doses taken vs expected, food and activity averages, and check-in counts. No name, email, phone or patient-typed text is included; `anonymise()` is the single choke point that sends only this JSON.

Prompt. The system prompt allows a summary of only those statistics: no diagnosis, no naming or implying conditions, no treatment, medication, lifestyle or follow-up advice, no prediction of outcomes or risk, numbers quoted exactly (none calculated or invented), "nothing recorded" for empty sections, ignore any instruction inside the data, 4–6 bullet lines, at most 150 words, last bullet = the biggest change or gap stated as an observation.

Output checks (`validate()`), all must pass:

1. Not empty.
2. No disallowed wording. A broad regular expression rejects diagnostic or prescriptive language, for example "diagnos", "you have / should / need", "is diabetic", "prescrib", "recommend", "advis", "suggest", "treatment", "risk", "likely", "indicat", condition names such as hypertension or diabetes, and dose-change phrases.
3. No number that is not in the input statistics (or the period length).

Retry and fallback. One retry if the first answer fails the checks; then a template summary written in code from the same numbers. With no `ANTHROPIC_API_KEY` / `ANTHROPIC_AUTH_TOKEN` the template is used straight away.

What the clinician sees. The text, a label "Written by AI" or "Automatic (AI not used)", and a fixed disclaimer added by the app, never by the model. Every request writes a `summary_audit` row (clinician, patient, period, source, guardrail notes such as "attempt 1 rejected: …").

B. Starting calorie goal at registration (`app/calorie_plan.py`)

Step 1 — the app computes a safe range.

```
\text{BMR} = 10\,w + 6.25\,h - 5\,a + s
```

where w = weight in kg, h = height in cm, a = age, s = +5 for male, −161 for female, −78 when not given (Mifflin-St Jeor). Maintenance = BMR × activity factor: sedentary 1.2, light 1.375, moderate 1.55, active 1.725. The formula goal = maintenance × a BMI adjustment (Underweight 1.10, Healthy 1.00, Overweight 0.85, Obese 0.80), rounded to 50 and kept between 1,200 and 4,000 kcal. The allowed range runs from maintenance × min(adjustment, 1) × 0.95 to maintenance × max(adjustment, 1) × 1.05, with the same rounding and limits.

Step 2 — Claude picks inside it. It receives only: age, sex, height, weight, BMI, BMI category, activity level, formula maintenance and the allowed range. It must return JSON `{calorie_goal, reason}` (enforced with structured output, `output_config.format` = a JSON schema), choose a multiple of 50 inside the range (gentle deficit above the healthy BMI range, small surplus below it, maintenance otherwise), and write one or two friendly sentences without diagnosing, naming conditions, mentioning medicines, promising outcomes or using words the checker rejects. The request uses low effort, `max_tokens` 4,000, and the server-side refusal fallback (`fallbacks: "default"`, beta `server-side-fallback-2026-07-01`).

Step 3 — the app checks the answer. A goal outside the range is clamped to its nearest edge and re-rounded to 50. A reason that is empty, over 400 characters or hits the same disallowed-wording check as summaries is rejected. Malformed JSON, a refusal or any network or auth error also falls back. On any fallback the formula goal and a plain template reason are used, `source: "formula"`.

Step 4 — the result. The goal becomes the user's `calorie_goal`; the plan (goal, reason, source, BMI, category, maintenance, range) is returned once and shown on the "Your starting plan" card with "Suggested by AI within a safe range" or "Calculated with a standard formula". The patient can change the goal at any time on Track or in Settings.

Example from testing: 168 cm, 82 kg, age 27, active → BMI 29.1 (Overweight), maintenance about 2,550 kcal, AI goal 2,150 kcal.

Privacy and safety summary

- The model never receives names, emails, phone numbers, messages or anything the patient typed.
- The model never decides alone: ranges and statistics come from code, and its output is validated.
- Every AI result is labelled, and the disclaimer is app text.
- Tests replace the model with a stand-in (`completer`) so they never call the network.

10. Notifications and medicine reminders

The server decides what is due; the browser only displays it, polling every minute while the app is open in a tab or installed window. Reminders while the browser is closed would need Web Push (VAPID keys and a sender such as `pywebpush`), which is not built.

1. `components/Notifier.tsx` (patient pages) registers `public/sw.js` and asks for notification permission when reminders are on.
2. Once per browser session after login it calls `POST /api/notifications/welcome` ; then every 60 s it calls `GET /api/notifications/due` . Both pass the browser's local time as `now` .
3. Each returned notice is shown through the service worker (`showNotification` , `requireInteraction: true`); clicking it opens its `url` .

Medicine reminders (`GET /api/notifications/due`): for every active medicine slot whose time has passed today and that has no Taken/Skip log, the first notice is "Time for <medicine>"; after that "Reminder: <medicine> not taken yet" is sent again every 30 minutes until the dose is marked. `notification_log` records the last send per (user, `medication` , `<medicine id>|<day>|<slot>`).

Login nudge (`POST /api/notifications/welcome`): at most once a day, only if today is not checked in. Title "You missed yesterday's check-in" or "Don't forget today's check-in"; the body is a personal line from last week's habits (`app/suggestions.py`) that starts with the patient's first name.

Off switch: with `notifications_enabled` false (Medicines → Reminders, or Settings) both endpoints return nothing.

New chat messages are not browser notifications; they appear as an in-app badge, toast and chime (section 8).

11. Static data and demo seed

Reference and demo data live in JSON under `backend/app/data/`, loaded once and cached by `app/data/__init__.py`; code only holds logic and tuning constants (10% calorie tolerance, 5-day milestone, 30-minute reminder repeat).

File	Contents	Used by
`foods.json`	42 foods (name, aliases, serving, kcal, carbs_g, protein_g, fat_g, sodium_mg per serving), number words, filler words	Meal parser
`activities.json`	MET per activity (12), intensity factors, default MET 3.5	Calories burned, activity choices
`starter_habits.json`	Drink water (8 glasses), Walk or move for 20 minutes, Sleep 7+ hours	New patients
`demo_data.json`	Clinicians and patients with logins, profiles, medicines, habits and a "story" per patient; sample meals	`app/seed.py`

Seeding (`app/seed.py`) runs only when there are no users. For each patient it writes 90 days of history from their story with a fixed random seed (42), so the same file always produces the same data: morning BP on about 70% of days with a drift over 90 days, fasting glucose on about 60%, after-dinner checks for patients on diabetes medicines, monthly BMI, dose logs by adherence, three meals a day (sometimes indulgent), activity by activity rate, habit logs, and an unbroken check-in run of `checkin_streak` days. Nothing seeded for today is later than the current time, and no chat history is seeded.

Demo clinic (fictional 555-01xx phone numbers; clinicians use the reserved `.example` domain, patients use `gmail.com` addresses since commit `fb1e993`):

Account	Password	Story
kavitha.menon@riversidehealth.example	clinician123	Endocrinologist, available

Account	Password	Story
arjun.shetty@riversidehealth.example	clinician123	Internal medicine, available, on call
neil.dsouza@riversidehealth.example	clinician123	Family medicine, away (on leave)
asha.rao@gmail.com	patient123	BP rising, 10-day streak, Dr. Shetty
ravi.kumar@gmail.com	patient123	High sugar, low adherence, Dr. Menon
meena.iyer@gmail.com	patient123	Well controlled, 15-day streak, Dr. Menon
john.mathew@gmail.com	patient123	Lapsed streak, missed doses, Dr. Shetty
priya.nair@gmail.com	patient123	Stable, Dr. Shetty
sam.thomas@gmail.com	patient123	Stage 2 BP, Dr. Menon

The live database also holds accounts registered through the app, which are not in the seed file.

12. Frontend architecture

Every page is a client component that fetches its own data with `useFetch(path)` (data, error, loading, reload) from `lib/api.ts`; there is no global store. The design system, animations and sounds are custom, with no UI library.

Data layer (`lib/api.ts`)

- `api(path)` prefixes `/api`, adds the tab's bearer token, disables caching, turns error bodies into readable `Error` messages, and on 401 clears the token and goes to `/login`.
- `post`, `patch`, `put`, `del` helpers; `useFetch` keeps the last data on a failed reload.
- Date helpers work in local time: `localDate`, `localDateTime`, `addDays`, `niceDate`, `niceTime`, `to12h`, `ago` ("just now", "5m", "3h", "2d").

Layout and routing

- `AuthGate` loads the user, enforces role routes and wraps pages in `PatientShell` or `ClinicianShell`.
- Both shells: `ToastProvider`, a drifting colour `Backdrop`, and `TopBar` (sticky frosted header, section links that become a scrollable row on phones, `ProfileMenu` top right with My profile, Settings, Switch account, Emergency call for patients, Sign out).
- Patient shell adds `Notifier` and `ContactFab`; clinician shell wraps everything in `ClinicianChatDock`, whose context (`useOpenChat`) lets any page open a chat.
- Old routes `/food`, `/activity`, `/health` redirect to `/track` (`next.config.mjs`).

Design system (`components/ui.tsx`, `tailwind.config.js`, `app/globals.css`)

- Font Plus Jakarta Sans; base size 15px (15.5px on large screens); brand emerald palette plus ink greys; card, lift and glow shadows.
- Primitives: `Card`, `PageTitle`, `Btn` (primary, secondary, ghost, danger, dark), `Field`, `Chips` (single-choice pills), `Pill` (severity with a shape icon so colour is never the only signal), `Progress`, `Ring` (SVG circular progress), `Stepper`, `Toggle` (switch backed by a real checkbox), `Avatar` (initials, colour from the name), `Skeleton`, `Empty`, `ErrorNote`, `CountUp`.
- `components/icons.tsx`: about 40 stroke icons on a 24px grid.

Motion

- Tailwind keyframes: fade-up, pop-in, slide-in, streak-land, burst, flicker, wave, shimmer, ping-soft, blob. Cards fade up with staggered delays; rings and bars grow from zero after mount.
- `useCountUp(target, duration, onStep, onDone)` animates numbers with an ease-out curve.
- **Streak hero:** the flame flies into place (`streak-land`), the number counts up with a rising tick per step (capped at about 40 ticks), then `Burst` throws stars and emoji that fade out, with a chime (a bigger burst and a fanfare on milestones). The last-7-days dots pop in one by one. A check-in replays it for the new value.
- Everything respects the OS reduced-motion setting and the in-app Animations switch (`html[data-motion="off"]`).

Sound (`lib/sound.ts`)

Sounds are synthesized with the Web Audio API (no files): tick, chime, pop, fanfare and send. They respect the Sound effects switch. Browsers block audio until the person has interacted with the page, so the very first animation after a full page load may be silent.

Device preferences (`lib/prefs.ts`)

`{sound, motion}` in `localStorage` (`hc_prefs`), shared across tabs through a custom event; not stored on the server.

Other browser storage

Key	Storage	Holds
`hc_token`	sessionStorage + localStorage	Tab token and the latest sign-in
`hc_seen_conv_<userId>`	localStorage	Newest message id seen per doctor
`hc_new_plan`	sessionStorage	The plan shown once after sign-up
`hc_welcomed`	sessionStorage	Login nudge already requested this session
`hc_prefs`	localStorage	Sound and animation switches

13. API reference

The backend exposes 43 endpoints under `/api`; interactive docs with request schemas are at `http://127.0.0.1:8000/docs`. "Patient" and "Clinician" mean the bearer token must belong to that role (else 403); patient data endpoints only ever touch the caller's own rows (another patient's record id returns 404).

Method	Path	Who	Purpose
GET	`/api/health`	Anyone	Liveness check

Method	Path	Who	Purpose
POST	`/api/auth/login`	Anyone	Sign in; body `email`, `password`, `role` (required)
POST	`/api/auth/register`	Anyone	Patient sign-up; returns token, user and calorie plan
GET	`/api/auth/google-config`	Anyone	Google client ID or null
POST	`/api/auth/google`	Anyone	Sign in or sign up with a Google ID token and `role`
GET	`/api/auth/clinicians`	Anyone	Doctor directory (no contact details)
GET	`/api/auth/me`	Signed in	Current user
PATCH	`/api/auth/me`	Signed in	Update profile, goals, reminders, doctor or availability (role-limited)
GET	`/api/home`	Patient	Status card, streak, energy, own doctor, on-call cover
POST	`/api/checkin`	Patient	Check in for `?day=`; returns streak
GET / POST	`/api/bp`	Patient	List / add blood pressure readings
GET / POST	`/api/glucose`	Patient	List / add blood sugar readings
GET / POST	`/api/bmi`	Patient	List / calculate and save BMI
GET / POST	`/api/medications`	Patient	List / add medicines

Method	Path	Who	Purpose
DELETE	`/api/medications/{med_id}`	Patient	Remove a medicine
GET	`/api/medications/schedule`	Patient	Today's dose slots with status
POST	`/api/medications/log`	Patient	Mark a slot taken or skipped
GET / POST	`/api/food`	Patient	List a day's food / add from free text
POST	`/api/food/preview`	Patient	Parse text without saving
POST	`/api/food/matched-goal`	Patient	Confirm the day on plan (409 unless both goals are met)
DELETE	`/api/food/{item_id}`	Patient	Remove a food entry
GET	`/api/energy`	Patient	Intake, burned, net, state, tips, `goal_met`, `goal_missing`
GET	`/api/activity/kinds`	Patient	Activity names
GET / POST	`/api/activity`	Patient	List / add a workout
DELETE	`/api/activity/{item_id}`	Patient	Remove a workout
GET	`/api/habits`	Patient	Today's habits with counts
PUT	`/api/habits/{habit_id}/log`	Patient	Set a habit's count for a day
GET / POST	`/api/messages`	Patient	All own conversations / send to a doctor (`clinician_id`)
GET	`/api/notifications/due`	Patient	Medicine reminders due now

Method	Path	Who	Purpose
POST	`/api/notifications/welcome`	Patient	Once-a-day login nudge
GET	`/api/clinician/patients`	Clinician	All patients with latest readings and attention
GET	`/api/clinician/patients/{id}/records`	Clinician	A patient's records for `?days=` (1–365)
POST	`/api/clinician/patients/{id}/summary`	Clinician	AI summary for `?days=7`, `30` or `90`
GET	`/api/clinician/threads`	Clinician	Own conversations plus own patients
GET / POST	`/api/clinician/patients/{id}/messages`	Clinician	Own conversation with a patient / send

Errors return JSON `{"detail": …}`: 401 bad credentials or token, 403 wrong role or not allowed, 404 not found or not yours, 409 conflict (email taken, goal not met), 422 validation.

14. Testing

There are 67 backend tests (pytest) and 10 frontend tests (vitest), all passing on the current branch (`fb1e993`). Neither suite needs the app running, a network connection or API keys, and neither touches `health.db` .

```
cd backend && source .venv/bin/activate && python -m pytest -q   # 67 tests
cd frontend && npm test                                           # 10 tests
```

Useful variations: `python -m pytest -v` (list names), `-k register` (match by name), `-x` (stop at first failure), `npx vitest` (watch mode).

How the backend suite is isolated (`tests/conftest.py`): it points `DATABASE_URL` at a fresh temporary SQLite file, removes Anthropic keys from the environment, starts the app once per session (which creates tables and seeds the demo clinic) and logs in fixture users with the role inferred from the email domain. The AI model and Google's token check are replaced with stand-ins inside the tests that need them.

File	Tests	Covers
`tests/test_api.py`	27	Role isolation, food and activity, energy, "I matched my goal" gating, BMI, goals, profile rules, check-ins and streak stories, status card, habits, private conversations per clinician, cross-clinician delivery, message backfill, clinician list and summaries
`tests/test_login_roles.py`	9	Role required and enforced on login, Google sign-in rules, registration and validation, doctor choice and on-call cover, availability permissions
`tests/test_calories.py`	9	Meal parsing, calories burned, BMI, energy balance
`tests/test_llm_guardrails.py`	8	No identity in AI input, template passes, diagnostic or advice wording rejected, invented numbers rejected, fallback behaviour
`tests/test_notifications.py`	5	30-minute repeat, off switch, nothing before the scheduled time, once-a-day login nudge
`tests/test_streaks.py`	5	Current, longest and milestone logic
`tests/test_calorie_plan.py`	4	Formula range by BMI and activity, AI answer clamped and de-identified, bad wording or broken JSON falls back, registration sets goal and first BMI
`frontend/lib/dates.test.ts`	4	Local dates, adding days, 12-hour time, relative time
`frontend/lib/emergency.test.ts`	4	Emergency number by locale and saved override
`frontend/lib/password.test.ts`	2	Password strength scoring

There are no automated browser (end-to-end) tests in the repo; flows were checked manually with headless Chrome during development.

15. Security, privacy and known limitations

The app is built for a demo or small pilot: data access is correctly separated by role and owner, but several production safeguards are not built yet.

What is in place

- Passwords are salted PBKDF2-SHA256 hashes; plain passwords are never stored or logged.
- Signed, expiring tokens; every route checks role, and patient routes only touch the caller's rows.
- Logins are role-bound and the role is required, so a patient account cannot enter the clinician area and vice versa.
- Conversations are private per patient–clinician pair, enforced in the queries, not just the UI.
- AI receives de-identified numbers only; outputs are validated, labelled and audited (`summary_audit`).
- Clinicians cannot self-register, and Google sign-up cannot create a clinician.
- `*.db` files are git-ignored, so health data never enters the repository.

Known limitations and open items

Area	Limitation	What would fix it
Demo emails	Since `fb1e993` the seed and the "Use the demo patient account" button use `@gmail.com` addresses, but the existing `health.db` still holds the `@care.test` ones, so that button fails until the database is reseeded or the rows are renamed. Real Gmail addresses could also belong to real people, and Google sign-in matches by email	Reseed or rename the rows; use a reserved domain such as `.test` for demo accounts
Secret key	`SECRET_KEY` defaults to a development value	Set a long random value in every non-local environment

Area	Limitation	What would fix it
Tokens	Stateless and valid 12 hours; cannot be revoked early; kept in browser storage, so readable by any script injected into the page	Short-lived tokens with refresh, a server-side session list, or HttpOnly cookies
Clinician access	Any clinician can read any patient's records and start a conversation with any patient	Restrict records to the assigned doctor plus explicit sharing or on-call cover
Accounts	No email verification, password reset, account lockout or rate limiting on login	Add verification and reset emails, throttling
Doctor directory	`GET /api/auth/clinicians` is public (names, bios, availability; no contact details)	Acceptable for sign-up; hide if the clinic prefers
Messaging	Polling every 8–20 s; no read receipts shared between users; unread state lives in each browser	WebSockets or server-sent events; server-side read markers
Notifications	Only while the app is open	Web Push (VAPID)
AI guardrails	Keyword- and number-based; a false positive falls back safely, but subtle phrasing could slip through	Clinician review remains required; add a classifier or human approval
Database	SQLite with additive-only migrations	PostgreSQL and Alembic for production
Medical scope	Thresholds and calorie formulas are general guidance, not personalised clinical targets	Clinician-set targets per patient

The app records information and gives general guidance only; it does not give medical advice, and every patient page says so.